

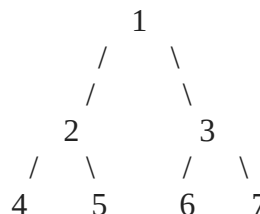
1. Show the output from the following sequence of priority queue ADT operations. The entries are key-element pairs, where sorting is based on the key value: insert(5,a), insert(4,b), insert(7,i), insert(1,d), removeMin(), insert(3, j), insert(6,c), removeMin(), removeMin(), insert(8,g), removeMin(), insert(2,h), removeMin(), removeMin()

Step 1: insert(5,a) => [(5,a)]
 Step 2: insert(4,b) => [(4,b), (5,a)]
 Step 3: insert(7,i) => [(4,b), (5,a), (7,i)]
 Step 4: insert(1,d) => [(1,d), (4,b), (7,i), (5,a)]
 Step 5: removeMin() => [(4,b), (5,a), (7,i)]
 Step 6: insert(3,j) => [(3,j), (4,b), (7,i), (5,a)]
 Step 7: insert(6,c) => [(3,j), (4,b), (7,i), (5,a), (6,c)]
 Step 8: removeMin() => [(4,b), (5,a), (7,i), (6,c)]
 Step 9: removeMin() => [(5,a), (6,c), (7,i)]
 Step 10: insert(8,g) => [(5,a), (6,c), (7,i), (8,g)]
 Step 11: removeMin() => [(6,c), (7,i), (8,g)]
 Step 12: insert(2,h) => [(2,h), (6,c), (7,i), (8,g)]
 Step 13: removeMin() => [(6,c), (7,i), (8,g)]
 Step 14: removeMin() => [(7,i), (8,g)]

2. Let T be a complete binary tree such that node v stores the key-entry pairs $(f(v), 0)$, where $f(v)$ is the level number of v . Is tree T a heap? Why or why not?

As T is a complete binary tree and a tree is only a heap if it is a complete binary tree, therefore, T is a heap.

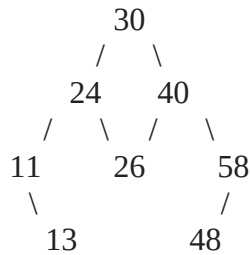
3. Is there a heap T storing seven distinct elements such that a preorder traversal of T yields the elements of T in sorted order? How about an inorder traversal? How about a postorder traversal?



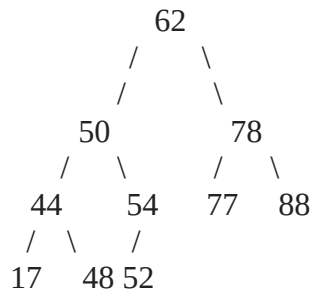
Consider the following heap T whose preorder traversal of T is 1, 2, 4, 5, 3, 6, 7, which is a sorted order of the elements in T . However, there cannot be a heap T storing seven distinct elements such that an inorder traversal of T yields the elements of T in sorted order. This is because the inorder traversal of a heap does not yield the elements in sorted order. To see why, consider the same heap T as above. The inorder traversal of T is 4, 2, 5, 1, 6, 3, 7, which is not a sorted order of the elements in T . Finally, there is no heap T storing seven distinct elements such that a postorder traversal of T yields the elements of T in sorted order. This is because the postorder traversal of a heap does not yield the elements in sorted order. To see why, consider the same heap T as above. The postorder traversal of T is 4, 5, 2, 6, 7, 3, 1, which is not a sorted order of the elements in T .

In conclusion, there exists a heap T storing seven distinct elements such that a preorder traversal of T yields the elements of T in sorted order, but there is no heap T storing seven distinct elements such that an inorder or postorder traversal of T yields the elements of T in sorted order.

4. Insert, into an empty binary search tree, entries with keys 30, 40, 24, 58, 48, 26, 11, 13 (in this order). Draw the tree after each insertion.



5. Draw the AVL tree resulting from the insertion of an entry with key 52 into the AVL tree of Figure 10.11(b)



6. Suppose S is a list of n bits, that is, n 0's and 1's. How long will it take to sort S with the merge-sort algorithm? What about quick-sort?

Sorting the list S using merge-sort has a time complexity of $O(n \log n)$. This is because the algorithm splits the list into two halves, each containing n bits of 0s and 1s, and merges them in linear time, or $O(n \log n)$. However, merge-sort is not aware of the fact that S contains only two possible values.

On the other hand, quick-sort with the three-way split method (pick, divide, and combine) will take $O(n)$ time to sort the list S , even when it contains only two possible values. The time complexity of merge-sort and quick-sort is $O(n \log n)$ in the average and worst cases, where n is the length of the list S . Therefore, it will take $O(n \log n)$ time to sort S with either merge-sort or quick-sort.

7. Show that the running time of the merge-sort algorithm on an n -element sequence is $O(n \log n)$, even when n is not a power of 2.

$$T(n) = T(n/2)[\text{to sort arr1}] + T(n/2)[\text{to sort arr2}] + n [\text{To merge both}]$$

Therefore, For $n = n$

$$T(n) = 2.T(n/2) + n$$

For $n = n/2$

$$T(n/2) = 2.T(n/4) + n/2$$

Or

$$T(n) = 2^2.T(n/2^2) + 2n$$

For $n = n/4$

$$T(n/4) = 2.T(n/8) + n/4$$

Or

$$T(n) = 2^3.T(n/2^3) + 3n$$

Deriving from these 3 cases, we get $T(n) = 2^i.T(n/2^i) + i.n$

When $T(1) = 1$

$$n/2^i = 1$$

$$\log n = i \log_2 n$$

$$i = \log_2 n$$

Put in $T(n)$

$$T(n) = 2^{\log_2 n} T(n/2^{\log_2 n}) + n \log_2 n$$

$$= nT(1) + n \log_2 n$$

$$T(n) = n + n \log_2 n$$

$$\text{Therefore } T(n) = O(n \log n)$$